

Package ‘imotionsApi’

September 21, 2026

Type Package

Title iMotions R library

Version 2.9.4.2

Date 2026-07-08

Description Provides functions to access your iMotions studies.

Copyright iMotions

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.4.0)

Imports jsonlite (>= 1.8.8),
httr (>= 1.4.7),
data.table (>= 1.15.4),
arrow (>= 15.0.1),
tidyr (>= 1.3.1),
dplyr (>= 1.1.4),
methods,
purrr (>= 1.0.2),
stringr (>= 1.5.1),
rlang (>= 1.1.3)

Suggests testthat (>= 1.0.2),
mockr (>= 0.1),
lintr (>= 1.0.0),
knitr,
markdown,
rmarkdown,
mockery

URL <https://imotions.com>,
<https://my.imotions.com/#studies>

VignetteBuilder knitr

Config/roxygen2/version 8.0.0

Contents

imotionsApi-package	2
convertRecordingTsToIntervals	3

createExport	4
getAoi	5
getAoiRespondentData	6
getAoiRespondentMetrics	7
getAoiRespondentTouchData	7
getAoiRespondentTouchMetrics	8
getAois	9
getRespondent	10
getRespondentIntervals	11
getRespondents	12
getSegment	13
getSegments	14
getSensorData	14
getSensors	15
getSensorsMetadata	16
getStimuli	17
getStimulus	17
getTouchActor	18
getTouchActorAoi	19
getTouchActorAois	20
getTouchActors	21
imConnection	22
imStudy	22
listLoadedStudies	23
listStudies	24
truncateSignalsByIntervals	24
unloadStudies	25
uploadAoiMetadata	26
uploadAoiMetrics	26
uploadEvents	27
uploadMetrics	28
uploadSensorData	30

Index	32
--------------	-----------

imotionsApi-package	<i>iMotions R API package</i>
---------------------	-------------------------------

Description

A client for accessing data from the iMotions Lab or iMotions Online platform.

Details

Use tokens (found on a remote study's R Analysis page or by right-clicking on an analysis in the software) to start accessing your data.

Author(s)

Maintainer: iMotions <support@imotions.com>

Authors:

- Amandine Grappe

Other contributors:

- Paolo Masulli [contributor]

See Also

Useful links:

- <https://imotions.com>
- <https://my.imotions.com/#studies>

Examples

```
myToken <- "xxxxxxx"
connection <- imotionsApi::imConnection(myToken)
```

convertRecordingTsToIntervals

Convert recording's timestamps (relative to data recording start) into stimulus/scene/AOI timestamps (relative to the interval first fragment start). Fragments are concatenated to give new array of timestamps in range [0, concatenated duration of stimulus/scene/AOI].

Description

Timestamps falling between an interval start/end will be kept, others will be discarded.

Usage

```
convertRecordingTsToIntervals(recordingTs, intervals, keepTs = FALSE)
```

Arguments

- | | |
|-------------|---|
| recordingTs | An array of recording's timestamps (relative to data recording start). Scalar, imSignals object as returned by getSensorData or data.table with a column Timestamp are also accepted. |
| intervals | An imInterval or imIntervalList object with start/end of a stimulus/scene/AOI as given by getRespondentIntervals or getAoiRespondentData . |
| keepTs | A boolean (or string) indicating whether timestamps falling outside an interval start/end should be kept unchanged, by default there are discarded. If keepTs is set to "NA", timestamps falling outside the interval will be replaced by NA. This will only work on array or scalar recordingTs. |

Value

A new array/scalar/data.table with timestamps in range [0, concatenated duration of stimulus/scene/AOI].

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ])
signals <- imotionsApi::getSensorData(study, sensors[3, ])
intervals <- imotionsApi::getRespondentIntervals(study, respondents[1, ])

# get a new signal with timestamps in range [0, concatenated duration of stimulus/scene/AOI].
signals <- imotionsApi::convertRecordingTsToIntervals(signals, intervals[1, ])

## End(Not run)
```

createExport	<i>Create an export file at a specific location and append metadata to it if provided. Note that a column with the study name will be appended to each export.</i>
--------------	--

Description

Create an export file at a specific location and append metadata to it if provided. Note that a column with the study name will be appended to each export.

Usage

```
createExport(
  params,
  study,
  data,
  outputDirectory,
  fileName,
  metadata = NULL,
  segment = NULL
)
```

Arguments

params	The list of parameters provided to the script.
study	An imStudy object as returned from imStudy .
data	A data.table containing the export metrics to save.
outputDirectory	The path where the file should be created.
fileName	The name of the file to create (should finish with .csv).
metadata	Optional - a data.table with metadata information. Column names will be converted to metadata headers and there must be a row corresponding to each data column.
segment	Optional - an imSegment object as returned from getSegment to upload the export. In case of a cloud study, this parameter needs to be provided.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
data <- data.frame("Respondent Name" = "Test", "Metric1" = seq(1:100), "Metric2" = rep(0, 100))
createExport(study, data, outputDirectory = "C:/Documents", fileName = "textExport.csv")

# Adding some metadata to the data
metadata <- data.table("Units" = c("", "ms", ""), "Description" = c("Desc1", "Desc2", "Desc"))
createExport(study, data, outputDirectory = "C:/Documents", fileName = "textExport.csv", metadata)

## End(Not run)
```

getAoi

Get a specific AOI from a study.

Description

Available AOIs can be found with [getAois](#). In case no AOI is found, return NULL.

Usage

```
getAoi(study, AOIId)
```

Arguments

study	An imStudy object as returned from imStudy .
AOIId	The id of the AOI you would like to retrieve.

Value

An imAOI object (data.table) containing the AOI of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
AOIs <- imotionsApi::getAois(study)
AOI <- imotionsApi::getAoi(study, AOIs$id[1])

## End(Not run)
```

getAoiRespondentData	<i>Get the inOutGaze information, inOutMouse information and AOI's intervals for a specific AOI/respondent combination. Note that imAOI object, by definition, are linked to a specific stimulus.</i>
----------------------	---

Description

The inOutGaze data.table has a IsGazeInAOI column that is TRUE when a gaze was recorded inside the AOI and FALSE if outside (timestamps correspond to the actual gaze point Timestamp). To reduce the size of the file created, only timestamps where a change of value occur are given. If the AOI was never active, the table is empty.

Usage

```
getAoiRespondentData(study, AOI, respondent)
```

Arguments

study	An imStudy object as returned from imStudy()
AOI	An imAOI object as returned from getAois .
respondent	An imRespondent object as returned from getRespondents .

Details

The inOutMouse data.table has a IsMouseInAOI column that is TRUE when a click was recorded inside the AOI and FALSE if outside (timestamps correspond to the actual Timestamp of each click). If no click was recorded or if the AOI was never active, the table is empty.

Value

A list with inOutGaze/inOutMouse information for the specific AOI/respondent combination and an imIntervalList object (data.table) composed of the start, end, duration, id and name of this AOI.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
AOIs <- imotionsApi::getAois(study)
respondents <- imotionsApi::getRespondents(study, AOI = AOIs[1, ])
AOIData <- imotionsApi::getAoiRespondentData(study, AOIs[1, ], respondents[1, ])

# Retrieving list items
inOutData <- AOIData$inOutData
intervals <- AOIData$intervals

## End(Not run)
```

`getAoiRespondentMetrics`*Get the metrics for a specific AOI/respondent combination.*

Description

Get the metrics for a specific AOI/respondent combination.

Usage

```
getAoiRespondentMetrics(study, AOI, respondent)
```

Arguments

<code>study</code>	An <code>imStudy</code> object as returned from <code>imStudy()</code>
<code>AOI</code>	An <code>imAOI</code> object as returned from getAois .
<code>respondent</code>	An <code>imRespondent</code> object as returned from getRespondents .

Value

A data.table of one row (`imMetrics` object) with metrics for the AOI /respondent combination of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
AOIs <- imotionsApi::getAois(study)
respondents <- imotionsApi::getRespondents(study, AOI = AOIs[1, ])
AOImetrics <- imotionsApi::getAoiRespondentMetrics(study, AOIs[1, ], respondents[1, ])

## End(Not run)
```

`getAoiRespondentTouchData`*Get the contact in/out data for a specific (touch actor, AOI)/respondent combination.*

Description

Get the contact in/out data for a specific (touch actor, AOI)/respondent combination.

Usage

```
getAoiRespondentTouchData(study, touchAoi, respondent)
```

Arguments

study	An imStudy object as returned from imStudy .
touchAoi	One row of getTouchActorAois , i.e. an AOI carrying a contact in/out fileId.
respondent	An imRespondent object as returned from getRespondents .

Value

A list with an inOutContact data.table (change-only rows of Timestamp, IsContactInAOI, Hand-Type, HandId) and an imIntervalList object with the AOI's active fragments. NULL if no in/out file was found.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimulus <- imotionsApi::getStimuli(study)[1, ]
respondent <- imotionsApi::getRespondents(study, stimulus = stimulus)[1, ]
touchAois <- imotionsApi::getTouchActorAois(study, stimulus, respondent)
contactData <- imotionsApi::getAoiRespondentTouchData(study, touchAois[1, ], respondent)

# Retrieving list items
inOutContact <- contactData$inOutContact
intervals <- contactData$intervals

## End(Not run)
```

```
getAoiRespondentTouchMetrics
```

Get the contact metrics for a specific (touch actor, AOI)/respondent combination.

Description

Get the contact metrics for a specific (touch actor, AOI)/respondent combination.

Usage

```
getAoiRespondentTouchMetrics(study, touchAoi, respondent)
```

Arguments

study	An imStudy object as returned from imStudy .
touchAoi	One row of getTouchActorAois , i.e. an AOI carrying a contact in/out fileId.
respondent	An imRespondent object as returned from getRespondents .

Value

A data.table of one row (imMetrics object) with the contact metrics, or NULL if none were computed yet.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimulus <- imotionsApi::getStimuli(study)[1, ]
respondent <- imotionsApi::getRespondents(study, stimulus = stimulus)[1, ]
touchAois <- imotionsApi::getTouchActorAois(study, stimulus, respondent)
metrics <- imotionsApi::getAoiRespondentTouchMetrics(study, touchAois[1, ], respondent)

## End(Not run)
```

getAois

Get AOIs from a study.

Description

Generic getAois function that takes as parameter a study object and optionally a respondent/stimulus object. In case no AOIs is defined for the combination, return NULL.

Usage

```
getAois(
  study,
  stimulus = NULL,
  respondent = NULL,
  generateInOutFiles = FALSE,
  verbose = FALSE
)
```

Arguments

study	An imStudy object as returned from imStudy .
stimulus	Optional - An imStimulus object as returned from getStimuli .
respondent	Optional - An imRespondent object as returned from getRespondents .
generateInOutFiles	Optional - A boolean indicating whether the corresponding InOut files should be generated and linked to each imAOI object.
verbose	Optional - A boolean indicating whether warnings from privateAOIfiltering should be shown.

Details

Important to note: to speed up the computation of gazes falling in/out AOIs at the respondent/stimulus level, an optional parameter generateInOutFiles can be enabled (both stimulus AND respondent arguments need to be provided). When enabled, a file will be generated for each AOI of this stimulus/respondent combination containing information regarding AOI activation/deactivation and gazes/clicks falling in. Filepaths to these newly generated files will be stored in their corresponding AOI object. If available, these filepaths will then directly be used by the [getAoiRespondentData](#) function instead of re-generating the files. This is particularly useful in case multiple AOIs are defined for the same stimulus. For remote studies, InOutFiles are already generated so this parameter doesn't apply.

Value

An imAOIList object (data.table) with all AOIs of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimuli <- imotionsApi::getStimuli(study)
respondents <- imotionsApi::getRespondents(study)

## Get all AOIs in the study
AOIs <- imotionsApi::getAois(study)

## Get all AOIs defined for a specific stimulus
AOIs <- imotionsApi::getAois(study, stimulus = stimuli[1, ])

## Get all AOIs defined for a specific respondent
AOIs <- imotionsApi::getAois(study, respondent = respondents[1, ])

## Get all AOIs defined for a specific stimulus/respondent combination
AOIs <- imotionsApi::getAois(study, respondent = respondents[1, ], stimulus = stimuli[1, ])

## Get all AOIs defined for a specific stimulus/respondent combination and process their InOut data
AOIs <- imotionsApi::getAois(study, respondent = respondents[1, ], stimulus = stimuli[1, ],
                             generateInOutFiles = T)

print(AOIs$fileId) # a field "fileId" should have been added with the path to the InOut file

## End(Not run)
```

getRespondent	<i>Get a specific respondent from a study.</i>
---------------	--

Description

Available respondents can be found with [getRespondents](#).

Usage

```
getRespondent(study, respondentId)
```

Arguments

study	An imStudy object as returned from imStudy .
respondentId	The id of the respondent you would like to retrieve.

Value

An imRespondent object (data.table) containing the respondent of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
respondent <- imotionsApi::getRespondent(study, respondents$id[1])

## End(Not run)
```

getRespondentIntervals

Get the list of time intervals (imIntervalList) for a given respondent.

Description

This imIntervalList is composed of stimuli/scenes/annotations intervals. For remote study, only stimuli intervals are supported as of now. Note that AOIs intervals can be retrieved using [getAoiRespondentData](#).

Usage

```
getRespondentIntervals(
  study,
  respondent,
  type = c("Stimulus", "Scene", "Annotation")
)
```

Arguments

study	An imStudy object as returned from imStudy .
respondent	An imRespondent object as returned from getRespondents .
type	The type of intervals to retrieve (can be set to Stimulus, Scene and/or Annotation).

Value

An imIntervalList object (data.table) composed of the start, end, duration, parent stimulus, id and name of each stimulus/scene/annotation. Annotations comments are also included.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
intervals <- imotionsApi::getRespondentIntervals(study, respondents[1, ])

# Get only the stimuli intervals
intervals <- imotionsApi::getRespondentIntervals(study, respondents[1, ], type = "Stimulus")

## End(Not run)
```

getRespondents	<i>Get respondents from a study.</i>
----------------	--------------------------------------

Description

Generic getRespondents function that takes as parameter a study object and optionally a stimulus/AOI/segment object. As AOIs are linked to a stimulus, it is not possible to provide both of them.

Usage

```
getRespondents(
  study,
  stimulus = NULL,
  AOI = NULL,
  segment = NULL,
  keepRespondentVariables = FALSE
)
```

Arguments

study	An imStudy object as returned from imStudy .
stimulus	Optional - An imStimulus object as returned from getStimuli .
AOI	Optional - An imAOI object as returned from getAois , or an imTouchAOI object as returned from getTouchActorAois to get the respondents that have contact data for that specific touch actor AOI instead of gaze data.
segment	Optional - An imSegment object as returned from getSegments .
keepRespondentVariables	Optional - A boolean indicating whether respondent variables should be kept (if they are available). If this has the default value of FALSE, only the "group" variable is exposed.

Value

An imRespondentList object (data.table) with all respondents of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimuli <- imotionsApi::getStimuli(study)
segments <- imotionsApi::getSegments(study)
AOIs <- imotionsApi::getAois(study)

## Get all respondents in the study
respondents <- imotionsApi::getRespondents(study)

## Get all respondents exposed to a specific stimulus
respondents <- imotionsApi::getRespondents(study, stimulus = stimuli[1, ])
```

```
## Get all respondents in a specific segment
respondents <- imotionsApi::getRespondents(study, segment = segments[1, ])

## Get all respondents for whom a specific AOI has been defined
respondents <- imotionsApi::getRespondents(study, AOI = AOIs[1, ])

## Get all respondents that have contact data for a specific touch actor AOI
respondent <- imotionsApi::getRespondents(study)[1, ]
touchAoi <- imotionsApi::getTouchActorAois(study, stimuli[1, ], respondent)[1, ]
respondents <- imotionsApi::getRespondents(study, AOI = touchAoi)

## Get all respondents in a specific segment exposed to a specific stimulus
respondents <- imotionsApi::getRespondents(study, stimulus = stimuli[1, ], segment = segments[1, ])

## Get all respondents in a specific segment for whom a specific AOI has been defined
respondents <- imotionsApi::getRespondents(study, AOI = AOIs[1, ], segment = segments[1, ])

## Get all respondents in the study and access their available variables
respondents <- imotionsApi::getRespondents(study, keepRespondentVariables = TRUE)

## End(Not run)
```

getSegment

Get a specific segment from a study.

Description

Available segments can be found with [getSegments](#).

Usage

```
getSegment(study, segmentId)
```

Arguments

study	An imStudy object as returned from imStudy .
segmentId	The id of the segment you would like to retrieve.

Value

An imSegment object (data.table) containing the segment of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
segments <- imotionsApi::getSegments(study)
segment <- imotionsApi::getSegment(study, segments$id[1])

## End(Not run)
```

getSegments	<i>Get all segments from a study.</i>
-------------	---------------------------------------

Description

Retrieves detailed information about segments in the study.

Usage

```
getSegments(study)
```

Arguments

study An imStudy object as returned from [imStudy](#).

Value

An imSegmentList object (data.table) containing all segments from the study.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
segments <- imotionsApi::getSegments(study)

## End(Not run)
```

getSensorData	<i>Download data corresponding to a specific sensor (signals/metrics).</i>
---------------	--

Description

Available sensors in your study can be listed using the [getSensors](#).

Usage

```
getSensorData(study, sensor, signalsName = NULL, intervals = NULL)
```

Arguments

study	An imStudy object as returned from imStudy .
sensor	An imSensor object as returned from getSensors .
signalsName	Optional - A vector of specific signals name you would like to return.
intervals	Optional - An imInterval or imIntervalList object with start/end of data to subset as given by getRespondentIntervals . In case of segment sensor, this is not possible.

Details

Signals always have a "Timestamp" column and are unique to a given respondent/segment and a given sensor source. Metrics are stored as a special sensor, also specific to a given respondent.

Value

An imData object (data.table) containing the signals (imSignals) or metrics (imMetrics) for the sensor of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ])
data <- imotionsApi::getSensorData(study, sensors[1, ])

## End(Not run)
```

getSensors

Get all sensors available for a given respondent/segment.

Description

Retrieves detailed information about sensors available for a given respondent/segment.

Usage

```
getSensors(study, target, stimulus = NULL)
```

Arguments

study	An imStudy object as returned from imStudy .
target	The target respondent/segment (an imRespondent/imSegment object as returned from getRespondents or getSegments).
stimulus	Optional - an imStimulus object as returned from getStimuli to retrieve sensors specific to this stimulus. In case of a segment target, this parameter needs to be provided.

Value

An imSensorList object (data.table) with all sensors collected for the respondent of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
segments <- imotionsApi::getSegments(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ])

# Get sensors for a specific stimulus
stimuli <- imotionsApi::getStimuli(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ], stimuli[1, ])

# Get sensors for a specific segment/Stimulus
stimuli <- imotionsApi::getStimuli(study)
sensors <- imotionsApi::getSensors(study, segments[1, ], stimuli[1, ])

## End(Not run)
```

getSensorsMetadata	<i>Get sensors specific metadata.</i>
--------------------	---------------------------------------

Description

Available sensors in your study can be listed using the [getSensors](#).

Usage

```
getSensorsMetadata(sensors)
```

Arguments

sensors An imSensorList object as returned from [getSensors](#).

Value

A data.table with sensors metadata (one row by sensor).

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ])
metadata <- imotionsApi::getSensorsMetadata(study, sensors)

## End(Not run)
```

getStimuli	<i>Get all stimuli from a study.</i>
------------	--------------------------------------

Description

Retrieves detailed information about stimuli in the study.

Usage

```
getStimuli(study, respondent = NULL, relevant = TRUE)
```

Arguments

study	An imStudy object as returned from imStudy .
respondent	Optional - An imRespondent object as returned from getRespondents .
relevant	A boolean indicating whether only relevant stimuli should be kept, by default non-relevant stimuli are discarded.

Value

An imStimulusList object (data.table) containing all stimuli from the study.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)

## Get all stimuli in the study
stimuli <- imotionsApi::getStimuli(study)

## Get all stimuli for a specific respondent
stimuli <- imotionsApi::getStimuli(study, respondents[1, ])

## End(Not run)
```

getStimulus	<i>Get a specific stimulus from a study.</i>
-------------	--

Description

Available stimuli can be found with [getStimuli](#).

Usage

```
getStimulus(study, stimulusId)
```

Arguments

study	An imStudy object as returned from imStudy .
stimulusId	The id of the stimulus you would like to retrieve.

Value

An imStimulus object (data.table) containing the stimulus of interest.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimuli <- imotionsApi::getStimuli(study)
stimulus <- imotionsApi::getStimulus(study, stimuli$id[1])

## End(Not run)
```

getTouchActor	<i>Get a specific touch actor from a stimulus.</i>
---------------	--

Description

Available touch actors can be found with [getTouchActors](#). Passing the result on to [getTouchActorAois](#) (instead of the whole stimulus) scopes the underlying contact detail fetch to just this actor - see [getTouchActorDetailsUrl.imTouchActor](#) - rather than triggering signal generation for every touch actor on the stimulus.

Usage

```
getTouchActor(study, stimulus, touchActorId)
```

Arguments

study	An imStudy object as returned from imStudy .
stimulus	An imStimulus object as returned from getStimuli .
touchActorId	The id of the touch actor you would like to retrieve.

Value

An imTouchActor object (data.table) containing the touch actor of interest, or NULL if none is defined for this stimulus or none matches touchActorId.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimulus <- imotionsApi::getStimuli(study)[1, ]
touchActors <- imotionsApi::getTouchActors(study, stimulus)
touchActor <- imotionsApi::getTouchActor(study, stimulus, touchActors$id[1])

## End(Not run)
```

getTouchActorAoi	<i>Get the touch actor AOI(s) matching a specific aoiId.</i>
------------------	--

Description

Calls [getTouchActorAois](#) and filters the result down to aoiId. Passing an imTouchActor pins the search to that one actor, so the result is a single imTouchAOI row. Passing an imStimulus can still return more than one row (an imTouchAOIList) if several touch actors share that interactive AOI.

Usage

```
getTouchActorAoi(study, imObject, aoiId, respondent = NULL)
```

Arguments

study	An imStudy object as returned from imStudy .
imObject	An imStimulus object as returned from getStimuli to look across every touch actor on it, or an imTouchActor object as returned from getTouchActor / getTouchActors to only look at that one actor.
aoiId	The id of the AOI you would like to retrieve.
respondent	Optional - An imRespondent object as returned from getRespondents to attach that respondent's contact in/out fileId/resultId to the result.

Value

An imTouchAOI object (data.table) if imObject narrows the match to a single (touch actor, AOI) combination, an imTouchAOIList if more than one touch actor shares aoiId, or NULL if none match.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimulus <- imotionsApi::getStimuli(study)[1, ]
touchAois <- imotionsApi::getTouchActorAois(study, stimulus)

touchActor <- imotionsApi::getTouchActor(study, stimulus, touchAois$touchActorId[1])
touchAoi <- imotionsApi::getTouchActorAoi(study, touchActor, touchAois$id[1])

## End(Not run)
```

getTouchActorAois	<i>Get the interactive AOIs of a stimulus's touch actors, optionally paired with their contact in/out data.</i>
-------------------	---

Description

Each touch actor marks one or more AOIs as interactive, and contact is recorded per (touch actor, AOI) pair. This returns one row per pair, plus the touch actor that produced it, so it can be used anywhere an imAOI is expected. It also carries the AOI's own definition (name, type, group, area), resolved from whichever camera the AOI is actually defined on (see [privateGetTouchActorAoiDefinitions](#)), so a touch actor on a non-Scene camera (e.g. Environment) is labelled the same as one on Scene.

Usage

```
getTouchActorAois(study, imObject, respondent = NULL)
```

Arguments

study	An imStudy object as returned from imStudy .
imObject	An imStimulus object as returned from getStimuli to return every touch actor's AOIs on it, or an imTouchActor object as returned from getTouchActors to return only that actor's AOIs.
respondent	Optional - An imRespondent object as returned from getRespondents to attach that respondent's contact in/out fileId/resultId to each row.

Details

Providing a respondent additionally attaches that respondent's contact in/out fileId/resultId to each row, triggering generation of any signal that is missing or older than the contact detection result it derives from. Without one, only the AOI/touch actor definitions are returned - use [getRespondents](#) (e.g. `getRespondents(study, AOI = touchAoi)`) to find which respondents have contact data for a given row.

Value

An imTouchAOIList object (data.table, a subclass of imAOIList so shared AOI functions still accept it) with one row per (touch actor, AOI) combination, or NULL if none exist. A single-row result is an imTouchAOI, subclassing imAOI the same way.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimulus <- imotionsApi::getStimuli(study)[1, ]

## List the touch actor AOIs defined for a stimulus, without triggering signal generation
touchAois <- imotionsApi::getTouchActorAois(study, stimulus)

## Get the respondents that have contact data for one of them
respondents <- imotionsApi::getRespondents(study, AOI = touchAois[1, ])
```

```
## Attach the contact in/out file for a specific respondent
touchAois <- imotionsApi::getTouchActorAois(study, stimulus, respondents[1, ])

touchActor <- imotionsApi::getTouchActors(study, stimulus)[1, ]
touchAois <- imotionsApi::getTouchActorAois(study, touchActor, respondents[1, ]) # only that actor's AOIs

print(touchAois$fileId) # path to the contact in/out file for each (touch actor, AOI) pair

## End(Not run)
```

getTouchActors

Get all touch actors defined for a specific stimulus.

Description

A touch actor detects hand-to-object contact on a recorded video and marks a set of AOIs as interactive. Contact detection runs in the iMotions desktop application only, so this returns NULL for a remote study.

Usage

```
getTouchActors(study, stimulus)
```

Arguments

study	An imStudy object as returned from imStudy .
stimulus	An imStimulus object as returned from getStimuli .

Value

An imTouchActorList object (data.table) with one row per touch actor, or NULL if none are defined.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
stimuli <- imotionsApi::getStimuli(study)
touchActors <- imotionsApi::getTouchActors(study, stimuli[1, ])

## End(Not run)
```

imConnection	<i>Create a connection with the iMotions API.</i>
--------------	---

Description

Tokens can be obtained by right-clicking on an analysis in iMotions and clicking "Get token for R API connection".

Usage

```
imConnection(token, baseUrl = NULL, s3BaseUrl = NULL, localPath = NULL)
```

Arguments

token	The token to be used for authentication.
baseUrl	Optional - The server to connect to in case of remote connection.
s3BaseUrl	Optional - The server to use to write back data in case of remote connection.
localPath	Optional - The path to write local file in case of remote connection (default to temporary folder).

Value

An imConnection object to be passed to other methods.

Examples

```
## Not run:
# Local connection
myToken <- "xxxxxxx"
connection <- imotionsApi::imConnection(myToken)

# Remote connection
myToken <- "token"
connection <- imotionsApi::imConnection(myToken, "myBaseUrl", "myS3BaseUrl", "myLocalPath")

## End(Not run)
```

imStudy	<i>Load an iMotions study by id.</i>
---------	--------------------------------------

Description

Retrieves detailed information about a study including stimuli, respondents, segments, and more.

Usage

```
imStudy(connection, studyId)
```

Arguments

connection An imConnection object as returned from [imConnection](#).
studyId The id of the study you would like to retrieve.

Details

Available studies can be found with [listStudies](#).

Value

An imStudy object to be passed to other methods.

Examples

```
## Not run:  
connection <- imotionsApi::imConnection("xxxxxxx")  
studies <- imotionsApi::listStudies(connection)  
studyId <- studies$id[1]  
study <- imotionsApi::imStudy(connection, studyId)  
  
## End(Not run)
```

listLoadedStudies	<i>List studies that have been loaded in the current session.</i>
-------------------	---

Description

More detailed information about a study can be retrieved using [imStudy](#).

Usage

```
listLoadedStudies()
```

Value

A list of the studies that have been loaded in the current session.

Examples

```
## Not run:  
connection <- imotionsApi::imConnection("xxxxxxx")  
studies <- imotionsApi::listStudies(connection)  
studyId <- studies$id[1]  
study <- imotionsApi::imStudy(connection, studyId)  
listLoadedStudies()  
  
## End(Not run)
```

listStudies	<i>List available studies.</i>
-------------	--------------------------------

Description

More detailed information about a study can be retrieved using [imStudy](#).

Usage

```
listStudies(connection)
```

Arguments

connection An imConnection object as returned from [imConnection](#).

Value

A data frame containing names and ids of studies.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)

## End(Not run)
```

truncateSignalsByIntervals	<i>Truncate signals data based on given intervals.</i>
----------------------------	--

Description

Any interval combination can be asked. Timestamps falling between an interval start/end will be kept, others will be discarded. If dropIntervals is set to TRUE, Timestamps falling between an interval start/end will be discarded.

Usage

```
truncateSignalsByIntervals(signals, intervals, dropIntervals = FALSE)
```

Arguments

signals	An imSignals object as returned by getSensorData or a data.table including a "Timestamp" column that needs to be cut.
intervals	An imInterval or imIntervalList object with start/end of data to subset as given by getResponseIntervals or getAoiRespondentData .
dropIntervals	A boolean indicating whether Timestamps falling between an interval start/end should be discarded, by default there are kept and other Timestamps are removed.

Value

A truncated imSignals object.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
sensors <- imotionsApi::getSensors(study, respondents[1, ])
signals <- imotionsApi::getSensorData(study, sensors[3, ])
intervals <- imotionsApi::getRespondentIntervals(study, respondents[1, ])

# get the 3 first intervals
dataSubset <- imotionsApi::truncateSignalsByIntervals(signals, intervals[1:3, ])

# remove the second interval
dataSubset <- imotionsApi::truncateSignalsByIntervals(signals, intervals[2, ], dropIntervals = TRUE)

## End(Not run)
```

unloadStudies

Remove studies that have been loaded in the current session.

Description

This function can be used when new changes have been made to already loaded studies.

Usage

```
unloadStudies()
```

Value

"Studies successfully removed from the current session" if success.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
studyId <- studies$id[1]
study <- imotionsApi::imStudy(connection, studyId)
unloadStudies()

## End(Not run)
```

uploadAoiMetadata	<i>Upload AOI metrics metadata for a specific study. The "Group" and "Group description" fields are mandatory as they are used to group the metadata.</i>
-------------------	---

Description

Upload AOI metrics metadata for a specific study. The "Group" and "Group description" fields are mandatory as they are used to group the metadata.

Usage

```
uploadAoiMetadata(study, metadata)
```

Arguments

study	An imStudy object as returned from imStudy()
metadata	A data.table with metadata information. Column names will be converted to metadata headers.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
metadata <- data.table("Units" = c("ms", ""), "Show" = c("FALSE", "TRUE"), "Group" = c("1", "2"),
  "Group description" = c("1", "2"))

uploadAoiMetadata(study, metadata)

## End(Not run)
```

uploadAoiMetrics	<i>Upload metrics for a specific respondent and AOI in a study.</i>
------------------	---

Description

Upload metrics for a specific respondent and AOI in a study.

Usage

```
uploadAoiMetrics(study, AOI, target, metrics)
```

Arguments

study	An imStudy object as returned from imStudy()
AOI	An imAOI object as returned from getAois , or an imTouchAOI object as returned from getTouchActorAois to upload metrics for that specific touch actor AOI instead.
target	The target respondent/segment for the sensor (an imRespondent/imSegment object as returned from getRespondents or getSegments).
metrics	A data.table containing the metrics to upload.

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
AOIs <- imotionsApi::getAois(study)
respondents <- imotionsApi::getRespondents(study, AOI = AOIs[1, ])
metrics <- data.frame("metric1" = 2, "metric2" = 234, "metric3" = 1234)
uploadAoiMetrics(study, AOIs[1, ], respondents[1, ], metrics)

## End(Not run)
```

uploadEvents	<i>Create events for a specific respondent in a study.</i>
--------------	--

Description

Events data.table must be composed of a EventName, Timestamp and Description column. Description will be rendered as tooltip in the software. Event upload is only supported for local connections. Remote connections are skipped.

Usage

```
uploadEvents(
  params,
  study,
  events,
  target,
  eventsName,
  scriptName,
  metadata = NULL
)
```

Arguments

params	The list of parameters provided to the script - specific parameters value will be stored as metadata.
study	An imStudy object as returned from imStudy .
events	A data.table containing the events to upload (imData object are also accepted).

target	The target respondent for the sensor (an imRespondent object as returned from getRespondents).
eventsName	The name of the new events you would like to create.
scriptName	The name of the script used to produce these signals.
metadata	Optional - a data.table with metadata information. Column names will be converted to metadata headers and there must be a row corresponding to each data column.

Details

Params required field are "iMotionsVersion" and "flowName" (flow name will be used to link events to the original script)

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)

events <- data.table("Timestamp" = seq(1:5), "EventName" = rep("My event names", 5),
                    "Description" = rep("Description of event", 5))

params <- list("iMotionsVersion" = 8, "flowName" = "Test")
uploadEvents(params, study, events, respondents[1, ], eventsName = "New events",
            scriptName = "Example Script")

# Adding some metadata to the events
metadata <- data.table("Units" = c("ms", "", ""), "Show" = c("FALSE", "TRUE", "TRUE"))
uploadEvents(params, study, events, respondents[1, ], eventsName = "New events",
            scriptName = "Example Script", metadata)

## End(Not run)
```

uploadMetrics	<i>Create metrics for a specific respondent in a study.</i>
---------------	---

Description

Metrics data.table must be composed of a StimulusId column, a Timestamp column, and at least one additional column with metrics. The Timestamp column should be filled with recording timestamps falling during the stimulus of interest (i.e. the timestamp of the start of the stimulus it corresponds to). Metrics upload is only supported for local connections. Remote connections are skipped.

Usage

```
uploadMetrics(
  params,
  study,
  metrics,
```

```

    target,
    metricsName,
    scriptName,
    metadata = NULL
  )

```

Arguments

params	The list of parameters provided to the script - specific parameters value will be stored as metadata.
study	An imStudy object as returned from imStudy .
metrics	A data.table containing the metrics to upload (imData object are also accepted).
target	The target respondent for the sensor (an imRespondent object as returned from getRespondents).
metricsName	The name of the new metrics you would like to create.
scriptName	The name of the script used to produce these signals.
metadata	Optional - a data.table with metadata information. Column names will be converted to metadata headers and there must be a row corresponding to each data column.

Details

Params required field are "iMotionsVersion" and "flowName" (flow name will be used to link metrics to the original script)

Examples

```

## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)

metrics <- data.table("StimulusId" = c("1000", "1001"), "Timestamp" = c(10, 20),
  "Metric1" = c(12, 25), "Metrics2" = c(NA_real_, 120))

params <- list("iMotionsVersion" = 8, "flowName" = "Test")
uploadMetrics(params, study, metrics, respondents[1, ], metricsName = "New metrics",
  scriptName = "Example Script")

# Adding some metadata to the data
metadata <- data.table("Units" = c("", "ms", "", "microSiemens"))
uploadMetrics(params, study, metrics, respondents[1, ], metricsName = "New metrics",
  scriptName = "Example Script", metadata)

## End(Not run)

```

uploadSensorData	Create a new sensor for a specific respondent/segment in a study.
------------------	---

Description

Signals data.table (with a Timestamp column) can be uploaded. After processing, the sensor can then be viewed and exported locally through the iMotions Desktop.

Usage

```
uploadSensorData(
  params,
  study,
  data,
  target,
  sensorName,
  scriptName,
  metadata = NULL,
  stimulus = NULL,
  overwrite = TRUE
)
```

Arguments

params	The list of parameters provided to the script - specific parameters value will be stored as metadata.
study	An imStudy object as returned from imStudy .
data	A data.table containing the signals to upload (imData object are also accepted).
target	The target respondent/segment for the sensor (an imRespondent/imSegment object as returned from getRespondents or getSegments).
sensorName	The name of the new sensor you would like to create.
scriptName	The name of the script used to produce these signals.
metadata	Optional - a data.table with metadata information. Column names will be converted to metadata headers and there must be a row corresponding to each data column.
stimulus	Optional - an imStimulus object as returned from getStimuli to upload data specific to this stimulus. In case of a segment target, this parameter needs to be provided.
overwrite	Optional - a boolean indicating if the new sensor should overwrite sensors generated for the same flowName. By default, they are overwritten.

Details

Params required field are "iMotionsVersion" and "flowName" (flow name will be use as "instance" of the new sensor)

Examples

```
## Not run:
connection <- imotionsApi::imConnection("xxxxxxx")
studies <- imotionsApi::listStudies(connection)
study <- imotionsApi::imStudy(connection, studies$id[1])
respondents <- imotionsApi::getRespondents(study)
segments <- imotionsApi::getSegments(study)
data <- data.frame("Timestamp" = seq(1:100), "Thresholded value" = rep(0, 100))
params <- list("iMotionsVersion" = 8, "flowName" = "Test")

uploadSensorData(params, study, data, respondents[1, ], sensorName = "New sensor",
  scriptName = "Example Script")

# Uploading data to a specific stimulus
stimuli <- imotionsApi::getStimuli(study)
uploadSensorData(params, study, data, respondents[1, ], sensorName = "New sensor",
  scriptName = "Example Script", stimulus = stimuli[1, ])

# Uploading data to a specific segment/stimulus
stimuli <- imotionsApi::getStimuli(study)
uploadSensorData(params, study, data, segments[1, ], sensorName = "New sensor",
  scriptName = "Example Script", stimulus = stimuli[1, ])

# Adding some metadata to the data
metadata <- data.table("Units" = c("ms", ""), "Show" = c("FALSE", "TRUE"))
uploadSensorData(params, study, data, respondents[1, ], sensorName = "New sensor",
  scriptName = "Example Script", metadata)

## End(Not run)
```

Index

convertRecordingTsToIntervals, 3
createExport, 4

getAoi, 5
getAoiRespondentData, 3, 6, 9, 11, 24
getAoiRespondentMetrics, 7
getAoiRespondentTouchData, 7
getAoiRespondentTouchMetrics, 8
getAois, 5–7, 9, 12, 27
getRespondent, 10
getRespondentIntervals, 3, 11, 14, 24
getRespondents, 6–11, 12, 15, 17, 19, 20, 27–30
getSegment, 4, 13
getSegments, 12, 13, 14, 15, 27, 30
getSensorData, 3, 14, 24
getSensors, 14, 15, 16
getSensorsMetadata, 16
getStimuli, 9, 12, 15, 17, 17–21, 30
getStimulus, 17
getTouchActor, 18, 19
getTouchActorAoi, 19
getTouchActorAois, 8, 12, 18, 19, 20, 27
getTouchActors, 18–20, 21

imConnection, 22, 23, 24
imotionsApi (imotionsApi-package), 2
imotionsApi-package, 2
imStudy, 4, 5, 8–15, 17–21, 22, 23, 24, 27, 29, 30

listLoadedStudies, 23
listStudies, 23, 24

privateGetTouchActorAoiDefinitions, 20

truncateSignalsByIntervals, 24

unloadStudies, 25
uploadAoiMetadata, 26
uploadAoiMetrics, 26
uploadEvents, 27
uploadMetrics, 28
uploadSensorData, 30